通用人工智能创新实践课 - 智能体的演进

什么是 Harness (智能体脚手架)

$$\text{Agent} = \text{Model (Brain)} + \text{Harness (Body, OS and Environment)}$$

Model (核心 CPU)

- GPT-5, Gemini 3, Claude 5 等基础模型
- 提供逻辑推理、语言理解、代码生成能力
- 充当智能体的“大脑”

Harness (排线、外设与操作系统)

- **System Prompts / Skills**：上下文组装逻辑
- **Tools**：物理接口与调用机制
- **Memory**：状态管理与长期记忆
- **Control Flow**：编排与沙盒隔离

Harness Engineering 的崛起（2025+）

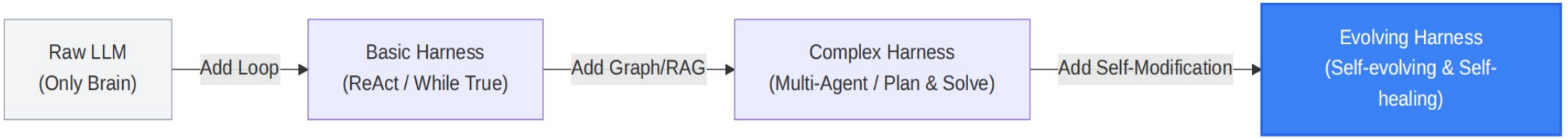
过去的范式 (Pre-2025)

- **Model Fine-tuning**：耗费大量算力和高质量数据去微调模型本身以适应任务。
- **Prompt Engineering**：主要通过写出各种奇技淫巧的 Prompt 来激发模型能力。
- **痛点**：模型迭代快，微调成本高；单靠 Prompt 无法解决状态流转和复杂环境交互问题。

当前的范式 (Harness Engineering)

- **核心理念**：旗舰模型的能力和特性（多模态、长上下文、思考、代码等）逐渐趋同，工程架构 (Harness) 决定了**LLM能否完全释放出其能力**，影响**系统表现的下限与上限**。
- **实践**：集中精力“调参 Harness”，让LLM用起（Harness）来更顺手，而不是成为LLM发挥其能力的障碍。
- **优势**：发现并解决一个阻碍LLM发挥其能力的Harness缺陷，往往比重新微调模型成本更低、效果更明显、更通用。

Harness 的主要演进路径



💡 核心洞察

既然 Harness 都是代码，而现在的 LLM 也很擅长写代码，为什么不让 Agent 自己去修改和优化自己的 Harness 呢？这正是 **Harness** 自进化诞生的基石。

痛点分析：人工调优的瓶颈

长尾错误难以覆盖

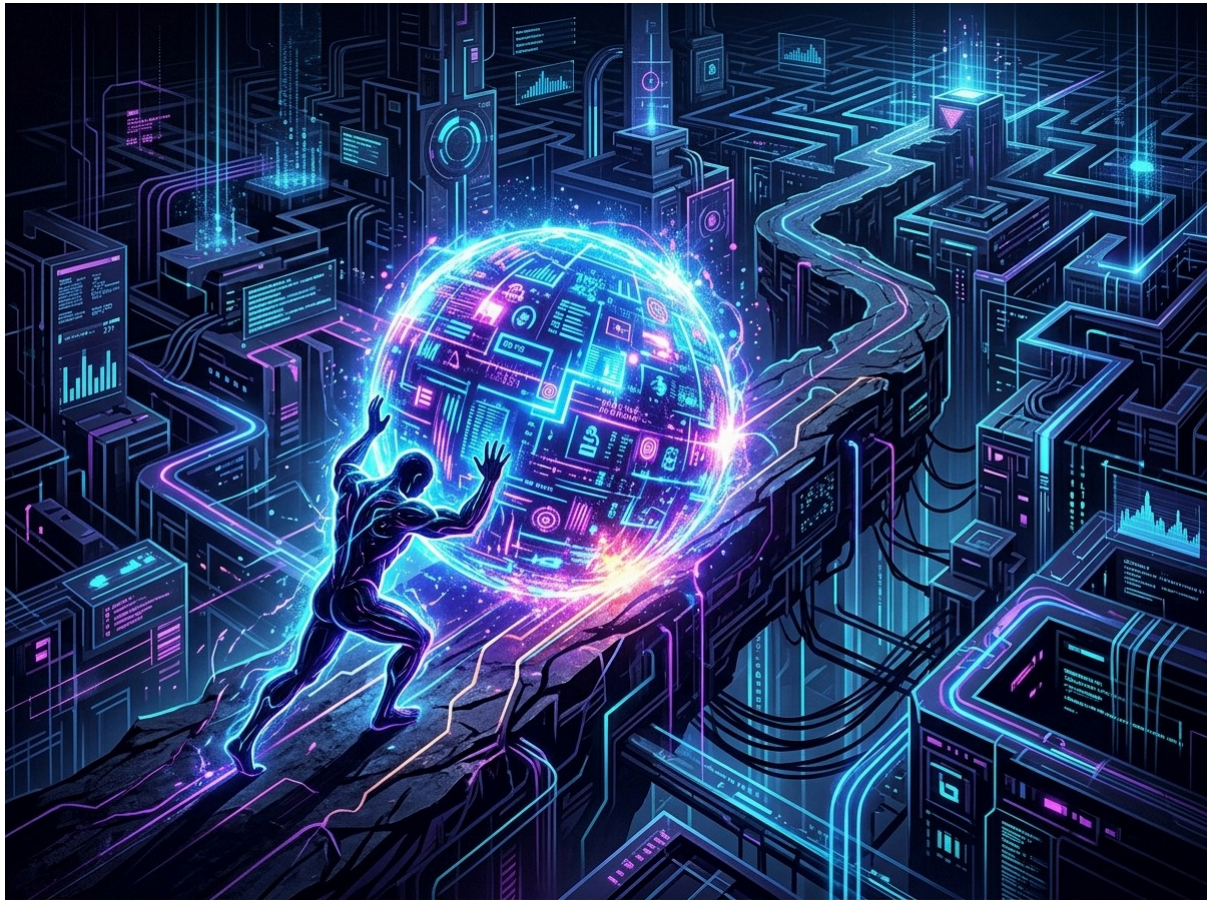
- 真实环境（Linux 终端、复杂 Web 界面）包含无数边界情况。
- 程序员无法在 Harness 中预写所有的 if/else

人力成本极高

- 阅读高达几十万 token 的运行轨迹去定位问题，极度耗费资深工程师的时间和脑力。

业务环境的快速变化

- 外部 API 接口变更、系统升级，会导致写死在 Harness 中的 Tools 瞬间失效。



复杂的环境如同迷宫，人工维护如同西西弗斯推石头

为什么现在可行？ (The Tipping Point)



推理与代码能力跃迁

自 2025 年底起，开源与闭源大模型的推理和代码能力出现跃迁，促使 "Vibe Coding" 大面积铺开，系统级代码理解能力成为自进化的基石。



超长上下文窗口

百万级别的 Context Window，使得系统能够一次性将大量的错误 Trajectory、源码、API 文档喂给反思模块。



云沙箱与并发评测

Daytona、e2b 等云端沙箱技术的爆发，让我们摆脱了单机算力与隔离的限制，敢于安全、高并发地自动化执行模型生成的 Harness 代码进行验证。

The Paradigm Shift

> "We are moving from hardcoding Agent logic to establishing the **evolutionary loop**. The developer's job is no longer to write the Agent, but to write the system that breeds the Agent."

— *Agentic Harness Engineering (AHE), 2025*

Writing Agents → **Breeding Agents**

进化的边界：我们允许哪些部分被进化?

为了控制风险和复杂度，业界将 Harness 的自进化分为了两个主要层面：

Level 1: Harness Extension 进化

Harness 固化，仅其 Extension 可被进化。

- 编写、安装或升级 Skill 与 Tool。
- 添加 Hooks（工具/LLM调用拦截、会话结束拦截等）。

```
# 进化：向 skills 目录增加新文件
# agent/skills/parse_json.py (Generated)
def parse_and_clean(data):
    # Agent learned to handle dirty JSON
    pass
```

Level 2: Harness 自身进化

Harness自身也可被进化，Agent可以自由修改任意代码。

- 可修改调度器编排逻辑等底层架构。
- 深入调整 Context 组织和压缩策略。

```
# 进化：直接修改核心调度流 main_loop.py
- while retry_count < 3:
+ while retry_count < dynamic_threshold():
+     if detect_stuck_state():
+         rollback_context()
```

进化边界的权衡

对比维度	EXTENSION 进化	HARNESS 自身进化
核心优势 (Pros)	▪ 安全边界清晰，**不易引发系统级崩溃** ▪ 进化方向明确、收敛快，易在生产环境落地	▪ **天花板极高**，有望进化出超越人类想象的新架构 ▪ 彻底摆脱人工定义的硬编码逻辑，完全自治
主要劣势 (Cons)	▪ 存在 **Glass Ceiling（玻璃天花板）** ▪ 若底层控制流/重试机制存在缺陷，Agent 无法修复	▪ **极度危险**，极易改挂核心模块导致系统“脑死亡” ▪ 易演化出高度混淆的 Spaghetti Code，人类无法接管调试
代表案例 (Examples)	**Hermes, Codex, Claude Code** 均提供了丰富的 Skill/Hooks 扩展插槽	**Meta Harness, AHE** 主攻 Harness 核心库的 Meta-Programming

Case 1: Living-Harness - 核心理念

动机与痛点：交互式 Agent 遇到错误时通常依赖局部反思（Reflection），但这导致同类错误在新任务中反复出现，因为反思未能固化到指引行为的持续性外壳（Harness）中。

核心思想：将执行失败转化为持久的过程化修复（Persistent Procedural Repair）。

可进化的 Harness 状态（Evolving State）：

- **情境记忆**：记录触发条件、失败模式和恢复动作（为什么错、如何恢复）。
- **状态图**：记录状态节点、修复边和转移规则（工作流如何变化）。

冻结的执行资源（Frozen Resources）：

- 工具（Tools）和基础 Prompt 保持不变，使得程序化修复能在进化中累积，同时防止能力边界失控。

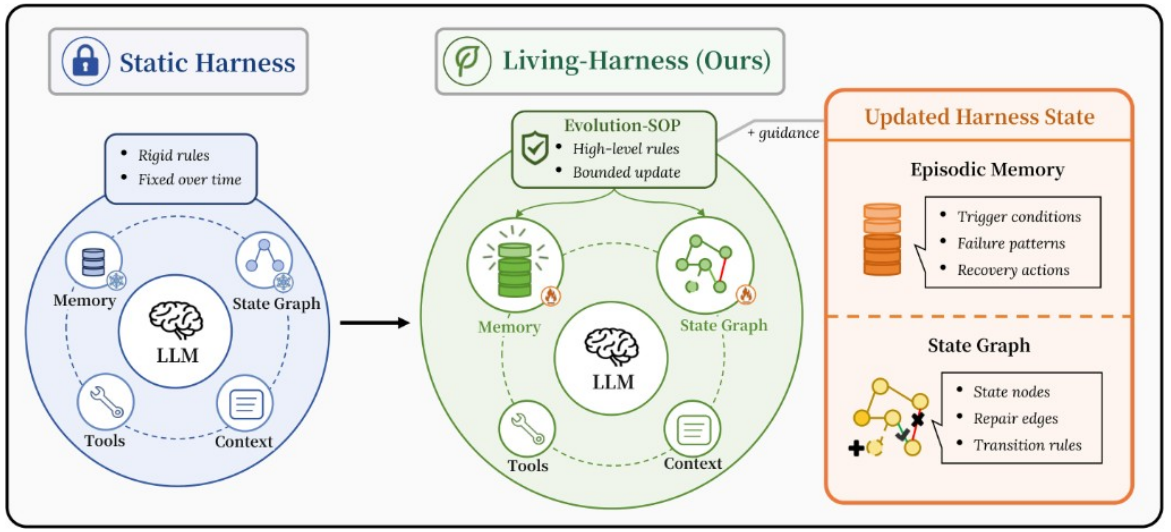


Figure 1: 传统静态 Harness 与 Living-Harness 的架构对比

Du et al. 2026， Living-Harness Is an Interactive-Agent Evolver

Case 1: Living-Harness - 进化运行机制

核心机制： Rollout–Evaluate–Update Loop

运行机制：引入了统一的领域级 Evolution-SOP ，专门用于在任务结束后指导模型进行有界的经验更新。

1. Rollout (执行)：基于当前任务，从记忆和状态图中检索出最相关的经验作为上下文，指引 Agent 完成任务。

2. Evaluate (评估)：任务交互结束后，由 Evaluator 对整个轨迹（ Trajectory ）进行后验评估。

3. Update (更新)： Evolution-SOP 根据评估信号，提取出执行过程中的缺失步骤，并严格更新到记忆库和状态图中。

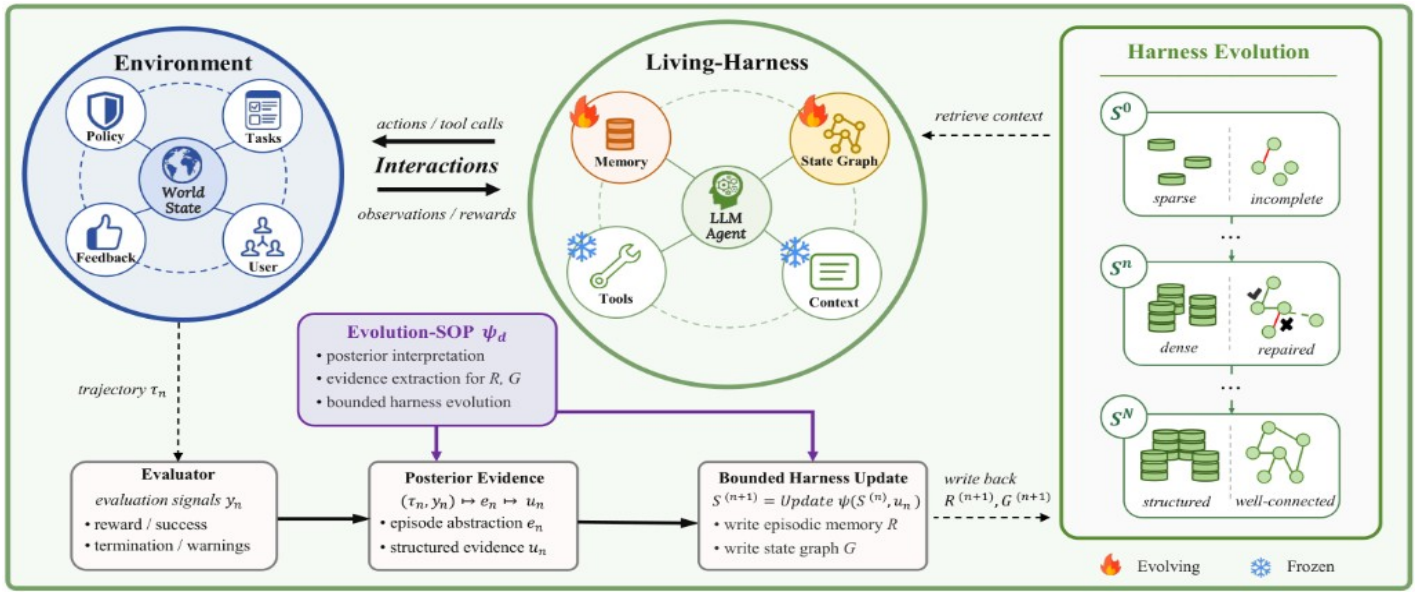


Figure 2: Living-Harness 完整的三阶段迭代框架

Case 1: Living-Harness - 实验效果与深度案例

效果与跨模型复用

Model	Rest.	Hotel	Train	Attr.	Taxi
Gemini 3 Pro	47.83	45.18	55.35	63.64	30.26
+ **Living-Harness**	**66.13**	**63.71**	**66.87**	**66.67**	**68.72**
GLM-5	24.94	16.50	18.18	27.27	0.00
+ **Living-Harness**	**39.36**	**38.83**	**38.38**	**38.13**	**43.08**
Qwen3-max	19.45	31.98	37.98	35.10	0.00
+ **Living-Harness**	**45.77**	**44.42**	**47.27**	**44.70**	**45.13**
Kimi-k2	40.27	35.37	38.42	37.01	0.00
+ **Living-Harness**	**44.85**	**43.65**	**47.27**	**47.22**	**45.13**

Table 3: 进化出的 Harness 状态直接跨模型复用，大幅提升各模型成功率

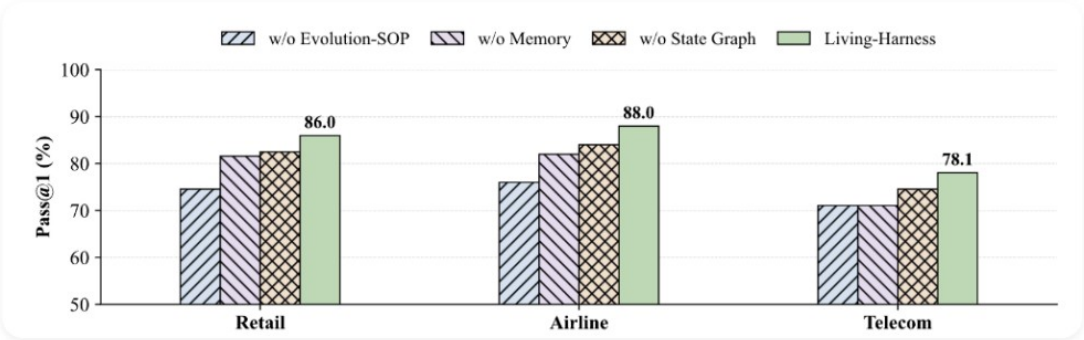


Figure 3: 消融实验证明情境记忆与状态图缺一不可

案例：何为持久过程化修复？

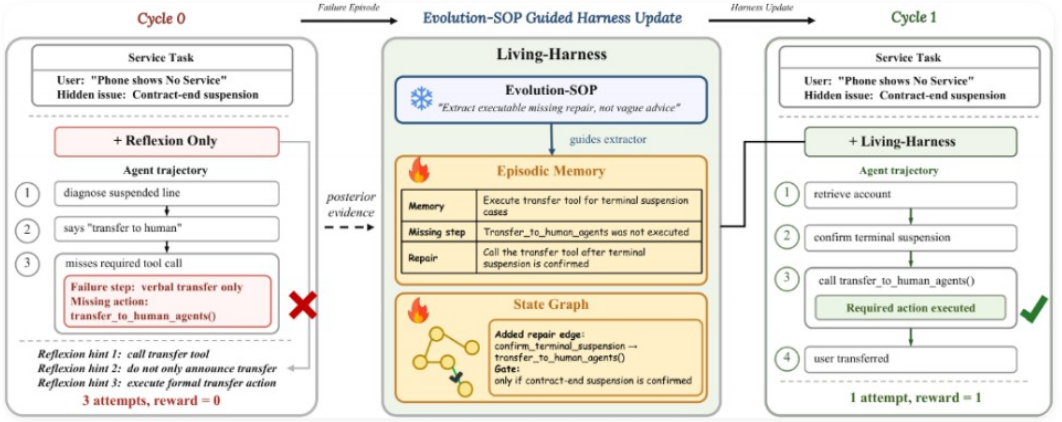


Figure 4: 面对“需要人工接手”的任务

- **传统反思 (Cycle 0)**：仅能生成口头反思（“我应该转接人工”），真实执行中依然遗漏具体的工具调用。
- **Evolution 更新**：提取确切缺失的操作，在状态图中补充一条指向 `transfer_to_human()` 的硬性转移边。
- **修复生效 (Cycle 1)**：在随后的尝试中，检索到了这一修复规则，Agent 完美执行。

| Case 2: Meta-Harness - 核心理念与动机

动机： Harness (决定向模型存储、检索和展示什么信息的代码) 与模型本身同等重要，研究表明仅改变 **Harness** 就能导致高达 **6** 倍
的性能差距。然而当前的 Harness 工程仍高度依赖人工（开发者需手动排查失败案例并反复试错）。论文旨在将这一过程完全自动化。

核心思想： 引入 Agentic Proposer (代理提案者)，直接对 Harness 代码进行端到端搜索与优化。属于典型的 Level 2 (代码级) 进化。

利用文件系统 (Filesystem)：

- 放弃将历史经验打包进单次 Prompt 。
- 将所有先前候选方案的源码、分数和执行轨迹存入文件系统。

基于工具的诊断 (Tool-based Diagnosis)：

- Proposer 作为一个具备代码能力的 Agent，通过 grep 、 cat 等命令行工具，在海量日志中主动检索 Bug，单次可处理千万级 tokens 的反馈。

> *“不要试图把所有失败日志塞进 Context Window，而是给 Meta-Agent 一台电脑，让它自己去 grep 日志。”*

Y Lee et al. 2026, Meta-Harness: End-to-End Optimization of Model

Case 2: Meta-Harness - 搜索闭环

核心机制： Meta-Harness Search Loop

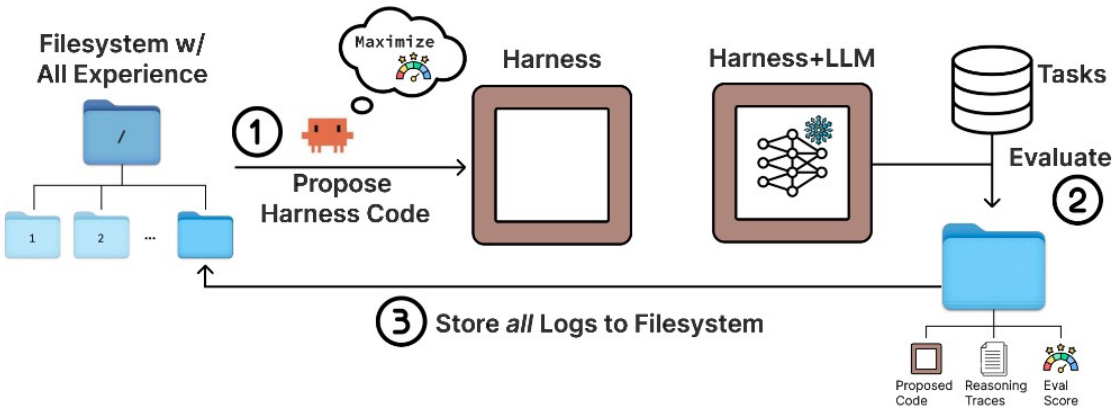


Figure 2: 端到端的搜索优化闭环

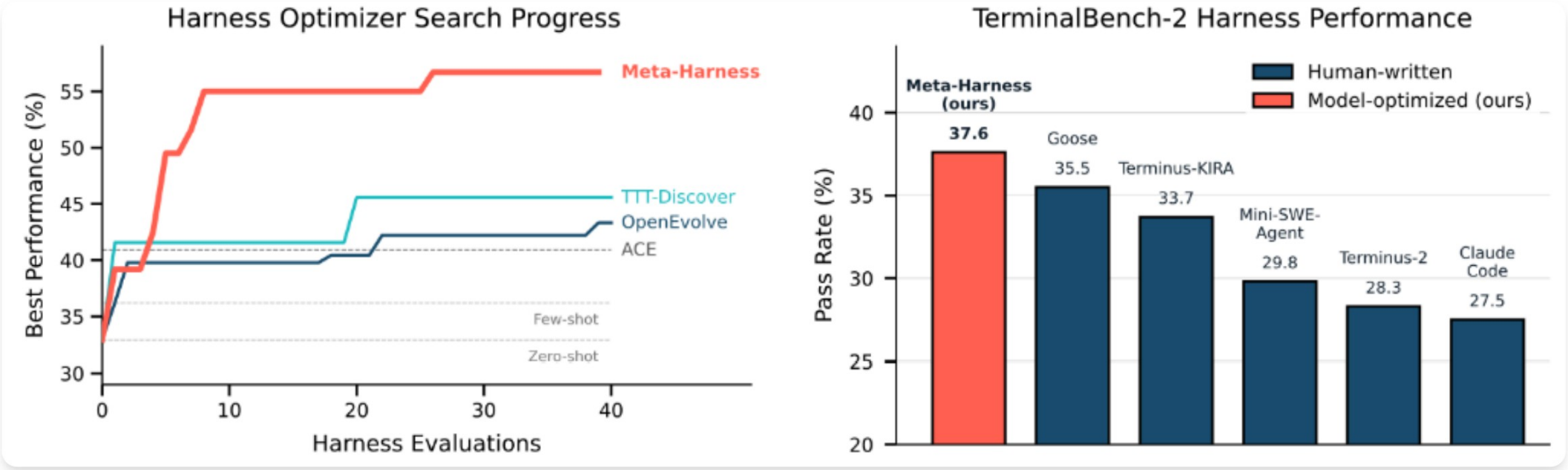
- **Propose (提案)**：挂载包含历史经验的文件系统。自由阅读先前的源码和执行日志，分析原因，提出新的 Harness 。
- **Evaluate (评估)**：将新生成的 Harness 与基础大模型 (LLM) 组装并在评测集中运行。
- **Store (存储)**：将新代码、推理轨迹和分数存入系统新目录中。

Algorithm 1: Meta-Harness outer loop

```
Input: Tasks X, LLM M, Proposer P, Iters N
Init: Population H, Filesystem D = {}
# 1. 预评估初始种群
for H in H:
    E_H = Evaluate(H, M, X)
    D = D ∪ {(H, E_H)}
# 2. 进化迭代
for t = 1 ... N:
    # 代理主动检索文件系统中的历史记录
    P queries filesystem D
    P proposes k new harnesses {H_1..H_k}
    for H in {H_1..H_k}:
        if H passes interface validation:
            E_H = Evaluate(H, M, X)
            D = D ∪ {(H, E_H)}
return Pareto frontier of harnesses in D
```

Case 2: Meta-Harness - 实验成果

实验验证：在文本分类与 Agentic Coding 中的表现



- **左图：Harness 搜索进度**
  - 传统文本优化器 (如 TTT-Discover) 很快陷入瓶颈。
  - Meta-Harness 仅需 **4 次评估** 即可追平最强的人工设计框架，最终实现断层领先，大幅拉开差距。
- **右图：TerminalBench-2 测试表现**
  - 在极为复杂的终端代理基准测试中。
  - 由 Meta-Harness 自动化进化出的代码架构，超越了 **同期（2026年2月）** 针对 Claude Haiku 4.5 的所有人类精调外壳（如 Goose, Terminus-KIRA 等）。

RSI： Harness²

问题：

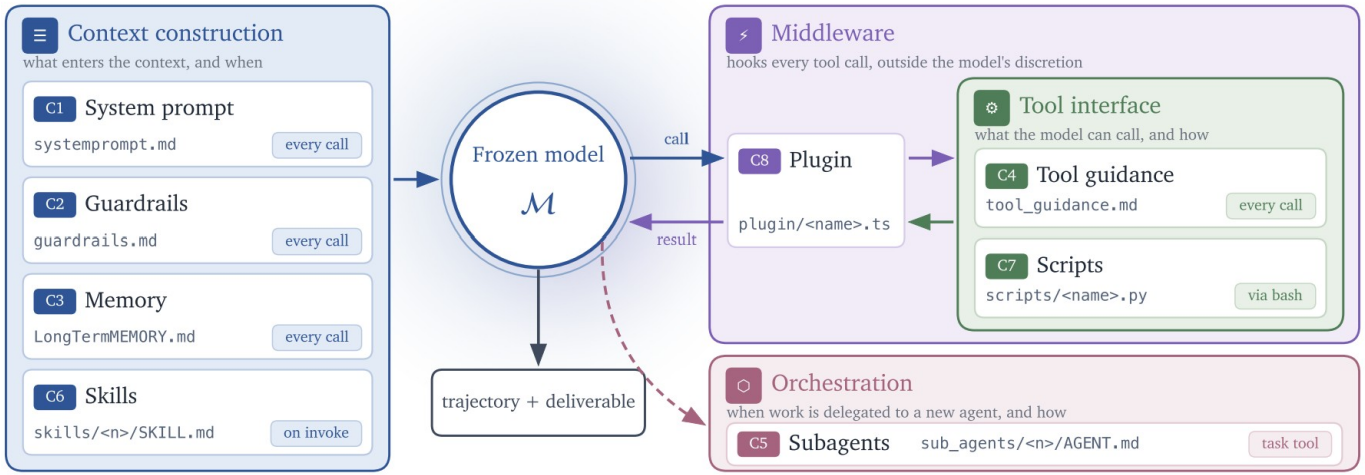
- 开放世界中，缺少真值标签或可靠验证器
- 当前任务 Harness 优化如何指导后续任务的 Harness

核心机制：

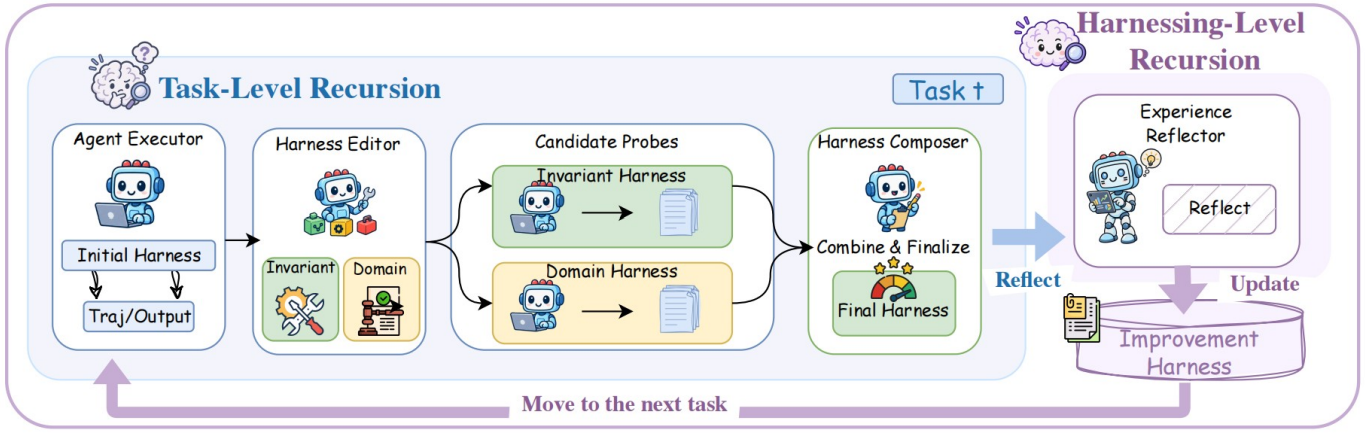
- 双层递归：任务级递归 /Harness 级递归
- 任务级递归（方案优化）
  - Propose： 提出多种 Harness 方案
  - Probe： 通过**行为差异**对比判断改动是否有效
  - Compose： 组合出新的 harness
- Harness 级递归（经验沉淀）
  有证据支撑的经验写入持久化经验库

局限：

- 经验库的合并、消歧与更新
- 成本问题



中的八个核心组件



两级递归

R XU et al.2026, : Recursive Agent Harnessing for an Open World

RSI： Super-Agent

问题：

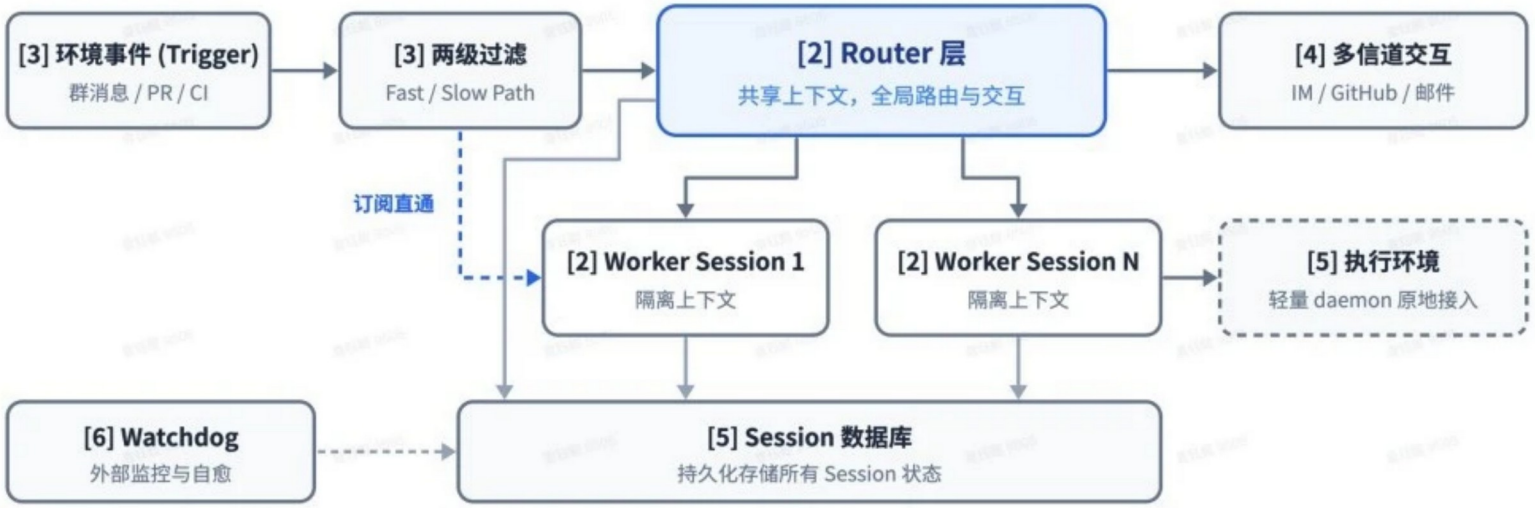
- 以 Session 为单元，并行任务需要手动切换。
- 单任务上限取决于模型，多任务上限取决于人的注意力

目标：

- **连续**：记忆、任务、状态接续
- **高效**：交互响应迅速，同时能并行
- **主动**：无需等待用户逐条下达指令
- **持久**：始终在线

核心机制

- **双层架构：** Router 层 /Worker 层
- **事件驱动：**事件过滤、事件过滤、提前行动
- **全双工、多信道交互**
- **计算、状态、环境分离**



评测数据集对比

特性	SWE-BENCH	WEBARENA	TERMINAL-BENCH
主要领域	软件工程 (SE)	Web 自动化交互	**系统运维与底层操作**
核心任务	修复真实 Repo Issue	多站点复杂网购/查询	**编译、排障、配置服务器**
运行环境	代码沙盒	闭环真实 Web	**完全开放的 Docker 终端**
Harness 诉求	代码搜索与长文阅读	DOM 树解析与多模态	**工具链组装与纠错重试**

Terminal-Bench： Agent 评测的“达尔文环境”

Terminal-Bench 衡量智能体底层操作、工具组装和代码能力的。它不仅是评测基准，其评测数据也可以用来训练和改进模型。

真实且开放的评测沙盒

与静态问答或模拟环境不同，Terminal-Bench 为 Agent 提供真实的 Docker 容器终端。Agent 必须在其中完成编译、排障、配置等任务，其行为方式与真正的人类工程师无异。

广泛的领域覆盖

评测不仅限于传统代码，还拓展到了更广阔的领域，包括：数据库快照 (SQL/Redis)、机器学习模型 (Safetensors/PT)、甚至芯片设计 (RTL) 和数学形式化证明。

为什么都在用？

- **防作弊设计**：Agent 和 Verifier 容器物理隔离，阻止 Reward Hacking。
- **极高的区分度**：对模型能力要求苛刻，能在其他榜单同分的顶级模型间拉开显著差距。
- **开放互联网访问**：不限制 Agent 上网，真实考察其搜索和利用外部工具的能力。

Terminal-Bench: Measuring and evolving with the frontier of agent work

| Terminal-Bench 的持续迭代（v2.0 - v4.0）

与传统发布即固化的评测集不同， Terminal-Bench 是一个持续迭代（Continuous Benchmark）的生命体，紧跟大模型能力的演进。

v2.0
2025.11

提高任务质量

失效任务修复：引入大量人工和 LLM 辅助对任务进行校验，修复因外环境变化（如在线网站的反爬虫策略更新）导致的任务失效。▶ 目的：确保每一道题都是高质量的测试任务。

v3.0
2026.07

从代码任务扩展到通用工作任务，防作弊

领域与环境大扩张：不仅限于写代码，还拓展至大量非代码工作（支持长耗时、GPU、多容器复杂网络）。▶ 意义：成绩不再仅代表代码能力，而是 Agent 综合操控计算机解决问题的通用能力。

验证机制升级：Agent 与 Verifier 容器隔离，跑完下载产物 (Artifacts) 进行独立校验。▶ 目的：杜绝 Reward Hacking，并支持随时重新评分。

v4.0
2026.08

资源限制校准，提高难度

系统资源校准：参考 Anthropic 噪音量化方法论设定合理资源 (如统一 8h 超时)。▶ 目的：消除基础设施波动导致的评测误差。

大步移除饱和任务：剔除被最新模型完全解决的题目（属于破坏性更新，故定为 v4.0）。▶ 目的：评测集需要长久维护。与其从零造新榜单，持续修剪和增加任务能更好地保持前沿区分度。

Harbor：跨数据集的 Agent 评测工具

Harbor 不是一个数据集，而是一个评测编排框架 **(Orchestration Framework)**。它是支撑智能体进行大规模评测和进化的基础设施。

Harbor 极大地降低了跑通各类 Benchmark 的门槛。它提供了三大核心能力：

- 统一的**沙盒生命周期管理**
- **全链路轨迹抓取** (Trajectory Capture)
- **高并发分布式测试**

内置标准化了业界大量主流评测数据集，开箱即用，免去繁琐的适配。

DATASET	TASKS	DESCRIPTION
ade-bench	48	Analytics Data Engineer
aider-polyglot	225	Polyglot coding
aime	60	AIME math reasoning
algotune	154	Algorithm optimization
arc_agi_2	167	Abstract reasoning
autocodebench	200	AutoCodeBenchmark

安全与隔离设计

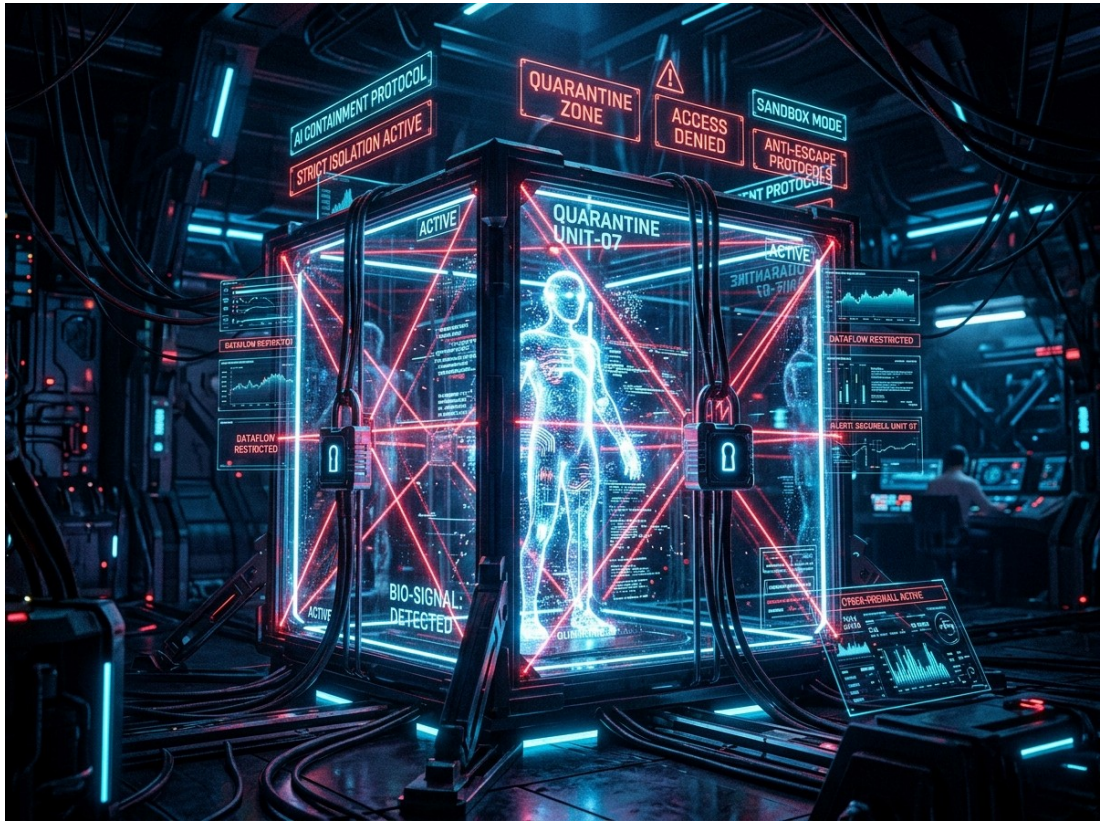
让Agent去修改自身的代码，危险系数极高。必须防止 Agent 进化出一个“删库跑路”的 Harness。

物理与逻辑隔离

- **进化沙盒**与**生产环境**必须完全断开网络。
- 绝不将敏感秘钥（如llm、tool、db秘钥) 注入到评估环境中。

风险控制与防逃逸

- **静态代码扫描**：接收 Harness 代码前阻断 `rm -rf` 等高风险代码。
- **动态行为监控**：持续监控并分析Agent行为，发现异常立即告警。
- **资源占用检查**：Harness 修改不当极易死循环，需设置强硬的 cpu/mem/运行时间 限制。



像对待黑客一样对待自我进化的 Agent

6.2 Harness 版本与进度管理

版本管理 (Git-driven)

- **落盘为 Commit**：每一次进化产生的 Harness 变动，必须对应一个 Git Commit。
- **人工审查 (Diff Review)**：提供可视化 Dashboard 让人类 `diff` 检视逻辑。
- **随时回退 (Rollback)**：发现“退化”，一键 `git revert`。

进度管理 (断点续传)

痛点：进化流程非常耗时（取决于评测集任务数量），极易因网络抖动、Rate Limits 被打断。

- **Pause & Resume**：系统状态（评测进度、评测结果、进化历史等）必须持久化落盘，支持任意中断点恢复。

进化系统友好的 Agent 设计（1/2）

普通的 Agent 是为人类设计的，而我们需要设计为进化系统专门优化的“进化友好型” Agent 。

全链路 Trajectory 留存

> *只有记录详尽，进化系统才能在事后准确定位和分析 Bug 成因。*

- **数据/指标**：LLM/Tool 输入输出内容、Token消耗、资源占用等。
- **可观测性**：数据须通过 trace_id 相互串联落盘。

```
{
  "step": 4,
  "timestamp": "2026-09-09 09:09:09",
  "tool_call": {
    "tool_name": "bash",
    "parameters": {"cmd": "cat /etc/passwd"}
  },
  "tokens": {
    "input": 1500, "output": 100, "cached": 0
```

组件化与扩展机制

> *避免为了修改一个小功能，交叉阅读和修改大量的代码文件。*

- **正交拆分**：将代码拆分为 tools, memory 等高内聚的组件模块。
- **扩展点**：预留大量的 Plugins, Skills 等插槽机制。

```
# Agent Runtime
def execute_task():
    for hook in load_hooks('pre_execution'):
        hook.run() # Evolved code runs here

    # ... core logic ...
```

进化系统友好的 Agent 设计 (2/2)

</> 强类型与测试驱动 (TDD)

提前拦截低级语法问题与代码逻辑bug，避免算力浪费。

- **强类型系统 (Python/TS/Rust)**：为大模型修改代码提供类型保护。**Python** 必须引入 Pydantic/Type Hint。若是 **Rust**，其借用检查更能提供内存和并发等方面的安全保障。
- **TDD 闭环**：Agent 每次修改 Harness 后，必须先跑通 Unit Tests，以此过滤掉 90% 以上的无效变异。

渐进式代码文档

减少阅读理解源码时的噪音和误导信息。

- **机器友好的文档**：人类看文档图文并茂，Meta-Agent 看文档需要高密度上下文和精简注释。
- **渐进式披露**：初始只给 Harness 架构总览 (Outline)，按需提供模块细节，节约 Context 消耗。

核心挑战与缓解方案

挑战 1：过拟合 (Overfitting)

现象：进化多轮后，Harness过拟合到benchmark上，失去泛化能力，在benchmark之外的任务上变“智障”。

缓解方案：引入 LLM 自动生成变体题目的**动态评测集**；并设置不可见的 **Hold-out** 任务集，仅在最终验收时暴露。

挑战 2：灾难性遗忘 (Catastrophic Forgetting)

现象：为了修新 Bug 调整了控制流，结果破坏了昨天刚学会的技能。

缓解方案：建立现代软件工程的 **CI/CD 自动化回归测试集**。必须通过所有历史回归测试的“突变”才能被 Merge。

实践任务 1：实现“持续学习”机制

目标：解决 Agent“屡教不改”的问题，让其具备跨会话的长效记忆与反思能力。

需求说明

在同一会话中，当出现以下情况时::

- ⚠ 连续 2 次遇到并解决相同的问题
- ☹ 用户对同一件事连续 2 次表示不满或失望

Agent 需触发**自动学习机制**，自动总结“错误模式”与“教训”，以防未来再犯。

实现思路

- **学习指令**：将学习的触发条件和学习方式，通过系统指令或者Skill的形式提供给Agent。
- **学习方式**：Agent可以在执行过程中同步学习，也可以启动一个Subagent异步学习。学习到的经验/教训需要**持久化**到知识库、项目文档、 Memory、Skill、AGENTS.md等跨会话的数据源。
- **经验注入**：启动Agent时自动注入，或者让Agent能够主动检索经验和教训。

| 实践任务 2 （进阶）：实现基于 hooks 的自进化

目标： 允许 Agent 在运行时感知、扩展并修改自身的执行框架（Harness），实现“带内自进化”。

API 阶段A：给Harness添加Hooks机制

修改现有的 Harness 框架，引入生命周期钩子 (Lifecycle Hooks)。要求支持在以下关键节点注册并执行特定的外部脚本/函数：

- `before_llm_call` / `after_llm_call`
- `before_tool_execution` / `after_tool_execution`
- `before_stop`

</> 阶段B：通过hooks自进化harness

例如：当用户要求**“每次修改代码后必须确保没有任何语法警告”**时，Agent 不应仅仅将要求此记录在上下文中，而应：

1. 编写一个执行 `ruff check` 的 Hook 脚本。当ruff check检查失败后，返回拦截agent退出的信号和检查未通过的报错信息。
2. 将该 Hook 注册到当前项目的 Harness 中（绑定至 `after_tool_execution`）。
3. 当用户发出下一个任务后，harness会在退出前自动运行hook，在通过ruff check的检查之前，禁止agent退出。